

General Crisis-Solving Brain — Blueprint

The General Crisis-Solving Brain — A deer-flow-style, Domain-Agnostic Blueprint

A graph-based, multi-agent engine that finds the root cause and a validated solution for any domain — Jordan Crisis Management Simulation Engine · 2026-05-31

0. What This Is (and Why It Is General)

This blueprint specifies a **domain-agnostic "brain"**: a graph-based, multi-agent flow — in the spirit of **ByteDance deer-flow** — that takes *any* case from *any* domain and (1) connects every signal and entity in a dependency graph, (2) correlates them into incidents, (3) finds the **root cause**, and (4) produces a **proper, validated solution**. It is **not** a single-domain app.

Core vs. Pack — the one idea that makes it general. The engine and the agent swarm never change. A domain plugs in *only* through a declarative **Domain Pack**: an ontology, propagation rules, data connectors, an intervention library, and a simulator adapter (§1). The same code that resolves a Zarqa water-main rupture resolves a cholera outbreak or a power-grid cascade — by swapping the pack. §5 proves this across **three** domains side-by-side.

Why deer-flow. deer-flow is a general LangGraph flow where a *coordinator* → *planner* → *research team* → *reporter* graph, with a human-in-the-loop review gate and pluggable MCP tools, solves arbitrary research tasks. We adopt that backbone and **re-skin it from research→report to diagnose→solve**: the worker team becomes graph-builder / correlator / root-cause / solution-generator / simulator-validator, and we add an adversarial **Critic** and a **human authorization gate** before any intervention ships (§2). *Implementation note from verification*: deer-flow v2.0 (Feb 2026) is now a dynamic "SuperAgent harness"; the classic plan→execute→synthesize **v1 graph** we reference as the pattern lives in its pre-v2.0 tagged releases / history, and is mirrored by LangGraph, `open_deep_research`, and `deepagents`.

A solution is "valid" iff (domain-independently) it (a) targets the **root cause**, not symptoms; (b) is **simulated** (via the pack's simulator) to reduce the incident's risk versus no action; (c) is **feasible** (resources, authority, time); (d) **bounds** second-order harm; and (e) carries a **confidence + evidence lineage** (§4).

The running example (one pack, not the product): the *Zarqa water-pipe cascade* — a trunk-main rupture (`PIPE-ZN-44`) cascades to hospital strain, traffic, and a 911 surge (+320%). The loud symptoms are the ER and the call centre; the **true root cause is the pipe**.

A valid fix acts on the water infrastructure (isolate + bypass + tanker stopgap), proven by re-simulating the cascade — not by adding ER staff.

How this document was produced. A 40-agent swarm: 9 framework-research agents (deer-flow + 8 classes of agentic frameworks) → 24 repo-verification agents → 6 design agents → 1 stack curator, on top of 30 previously-verified engine libraries. **24 frameworks + 30 libraries verified as real and maintained.**

Reading map: §1 architecture & the Domain-Pack interface · §2 the deer-flow-style solver swarm · §3 the root-cause→solution engine · §4 validation & safety · §5 the three-domain proof · §6 the verified tech stack · §7 build plan & adopting deer-flow.

1. General Brain Architecture & the Domain-Pack Interface

The brain is a **fixed engine** wrapping a generic **Crisis Dependency Graph (CDG)** and a fixed processing loop. Domains never touch the engine; they plug in through one declarative **Domain Pack**. This separation is the whole thesis: swap the pack, keep the brain.

1.1 What is CORE (never changes)

The CORE owns the graph substrate, the orchestration loop, and the generic algorithms (entity resolution, correlation, root-cause search, risk cascade, solution scoring). It speaks only in **generic types** — `Node`, `Edge`, `Signal`, `Incident`, `CauseHypothesis`, `Solution` — and never imports a domain symbol.

```
# CORE generic graph types — domain-agnostic, fixed forever
class Node(BaseModel):
    id: str # stable entity id, e.g. "PIPE-ZN-44"
    type: str # PACK-declared ontology label, opaque to core
    attrs: dict[str, float | str | bool]
    ts: datetime | None = None

class Edge(BaseModel):
    src: str; dst: str
    type: str # PACK label (e.g. "supplies", "infects")
    weight: float # propagation strength [0,1]
    lag: timedelta # propagation delay
    attrs: dict[str, float] = {}

class Signal(BaseModel): # one observation from a connector
    entity_id: str; metric: str
    value: float; ts: datetime; source: str
```

The fixed loop (each step is a generic function over generic types):

Ingest → Resolve → Correlate → RootCause → Risk → GenerateSolution →
Validate → Recommend → Learn

`Ingest` pulls `Signal`s from pack connectors; `Resolve` merges signals to graph `Node`s (entity resolution); `Correlate` clusters co-moving/causally-lagged signals into `Incident`s; `RootCause` walks edges *upstream* against propagation lag to rank `CauseHypothesis` (separating the loud symptom from the origin); `Risk` runs forward cascade to score blast radius; `GenerateSolution` (§3) maps the top cause-type to intervention templates; `Validate` (§4) simulates each candidate; `Recommend` ranks by the valid-solution criteria; `Learn` writes outcomes back to tune edge weights. See §2 for the agent swarm that executes these steps and §5 for the multi-domain proof.

1.2 What is PACK (per-domain, declarative only)

A pack supplies five members and **zero control flow**. The brain calls them; they never call the brain.

```
class DomainPack(Protocol):
    name: str

    # (i) ONTOLOGY – declares legal node/edge types + maps raw → generic
    # graph
    def ontology(self) -> Ontology: ...          # node_types, edge_types
    def to_nodes(self, raw: dict) -> list[Node]: ...
    def to_edges(self, raw: dict) -> list[Edge]: ...

    # (ii) PROPAGATION – edge weight/lag semantics OR a learned model
    def propagate(self, edge: Edge, x: float) -> float: ... # downstream
    # effect
    # falls back to learned GNN if rule returns None

    # (iii) CONNECTORS – signal adapters (pull/stream external data →
    # Signal)
    def connectors(self) -> list[Connector]: ...

    # (iv) INTERVENTION LIBRARY – cause-type → countermeasure templates
    def interventions(self) -> dict[str, list[InterventionTemplate]]: ...

    # (v) SIMULATOR – adapter used by §4 to validate a Solution
    def simulator(self) -> Simulator: ...          # apply(solution)-
    # >RiskDelta
```

Supporting pack-side types are also declarative:

```
class InterventionTemplate(BaseModel):
    id: str; cause_type: str
    action: str; params_schema: dict
    cost: float; authority: str; eta: timedelta

class Connector(Protocol):
    def poll(self) -> list[Signal]: ...
```

1.3 Two packs against the same interface

```
water = DomainPack(                                     # Zarqa worked
    example
    name="water",
    ontology=Ontology(
        node_types=["TrunkMain","Junction","Hospital","Sensor","CallCenter"],
        edge_types=["supplies","strains","routes_to","reports"]),
    propagate=lambda e,x: >*e.weight if e.type=="supplies" else
        x*e.weight*0.6,
    connectors=[ScadaConnector(), CAD911Connector(), HospitalEDConnector()],
    interventions={
        "trunk_main_rupture":
            [Template("isolate_valve",cost=2,authority="utility",eta="20m"),

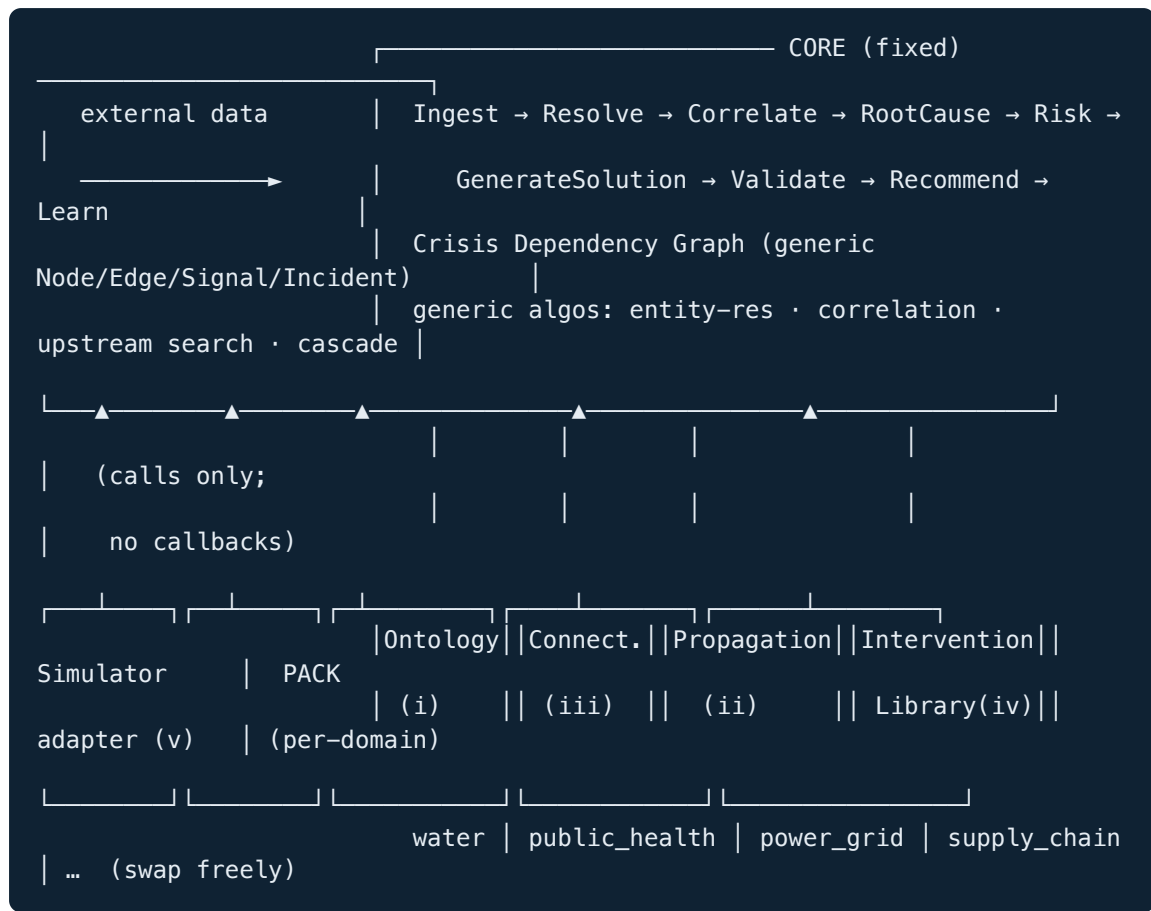
              Template("reroute_zone",cost=5,authority="utility",eta="45m")]},
    simulator=EpanetHydraulicSim())

public_health = DomainPack(                             # SAME core,
    different pack
    name="public_health",
    ontology=Ontology(
        node_types=["Population","Clinic","WaterSource","Vector","Lab"],
        edge_types=["infects","treats","contaminates","reports"]),
    propagate=lambda e,x: seir_step(x, e.weight, e.lag), # learned/epi model
    connectors=[LabResultConnector(), SyndromicConnector(), WWTPConnector()],
    interventions={
        "point_source_contamination":
            [Template("boil_water_notice",cost=1,authority="moh",eta="2h"),

              Template("source_shutoff",cost=4,authority="moh",eta="6h")]},
    simulator=SEIRCompartmentSim())
```

Same `RootCause`, `Risk`, and `Validate` code runs over both: water isolates `PIPE-ZN-44` upstream of the 911 surge; public-health isolates a contaminated `WaterSource` upstream of the clinic spike. The engine diff is **zero lines**; only the five pack members differ. The §6 tech stack (LangGraph) hosts the loop; §7 covers the build order; §5 runs both packs end-to-end as the proof.

1.4 Core-vs-Pack boundary (ASCII)



Everything above the line is written once. Everything below is a declarative pack. A new domain ships by authoring five members — never by editing the brain.

2. The General Solver Swarm (deer-flow-style)

The swarm is a **domain-agnostic** LangGraph flow. It maps deer-flow's `coordinator` → `planner` → `research-team` → `reporter` + `human-in-the-loop` onto a flow that **diagnoses and fixes** instead of researches. **Core = the nodes, the state schema, the routing. Pack = the ontology/rules/connectors/intervention-library/simulator the nodes read** (see §1 Architecture). No node hardcodes a domain; every domain-specific fact is pulled from `state.pack`.

2.1 Shared state (graph + blackboard)

All agents are pure functions `state -> partial_state` over one typed `SolveState`. The **graph** is the entity/edge substrate; the **blackboard** is the evolving analysis the agents read/write.

```

class SolveState(TypedDict):
    case_id: str
    pack: DomainPack          # ontology, rules, connectors,
                             # interventions, simulator (PACK)
    raw_signals: list[Signal]  # ingested, pre-resolution
    graph: EntityGraph        # nodes=entities, edges=typed deps
                             # (shared, CORE)
    incidents: list[Incident]  # correlated clusters
    hypotheses: list[CauseHypothesis] # candidate root causes + scores +
    # evidence lineage
    root_cause: CauseHypothesis | None
    solutions: list[SolutionPlan]
    validated: list[ValidationResult] # simulator output per solution
    critiques: list[Critique] # red-team objections, each open/resolved
    confidence: float          # 0..1, gate threshold
    loop_count: int
    decision_card: DecisionCard | None
    gate: Literal["pending", "approved", "rejected", "revise"]

```

`CauseHypothesis` and `SolutionPlan` always carry `evidence: list[GraphRef]` (node/edge ids) so every claim is traceable to the graph — this lineage is what the Critic and Human-Gate inspect.

2.2 Roles (input → output)

Agent	Reads	Writes	deer-flow analog
Coordinator	<code>raw_signals</code> , case meta	selects <code>pack</code> , normalizes intake, routes	coordinator
Planner	graph stub, incidents	a solve-DAG (which workers, what order, stop criteria)	planner
Graph-Builder	<code>raw_signals</code> , <code>pack.ontology+connectors</code>	<code>graph</code> (entity- resolved, typed edges)	researcher
Correlator	<code>graph</code> , <code>pack.rules</code>	<code>incidents</code> (clustered signals)	researcher
Root-Cause Analyst	<code>graph</code> , <code>incidents</code> , <code>pack.rules</code>	ranked hypotheses , sets <code>root_cause</code>	researcher
Solution-Generator	<code>root_cause</code> , <code>pack.interventions</code>	<code>solutions</code> (countermeasures + resources)	coder
Simulator/Validator	<code>solutions</code> , <code>pack.simulator</code>	<code>validated</code> (Δ risk vs no-action, 2nd-order harm)	coder
Critic / Red-Team	<code>root_cause</code> , <code>solutions</code> , <code>validated</code>	<code>critiques</code> ; may reopen the loop	(ADDED)
Human-Gate	<code>decision_card</code>	<code>gate</code> $\in$ approve/reject/revise	human-in- the-loop
Reporter	full blackboard	<code>decision_card</code> (Insight card, §4 Validation)	reporter
Memory	outcome vs prediction	updates <code>pack</code> priors / hypothesis weights	(ADDED)

REUSE from deer-flow: the coordinator→planner→team→reporter skeleton; the human-interrupt pattern; `Command(goto=..., update=...)` handoffs; MCP tool binding for connectors; per-node prompt+structured-output convention. **ADD:** the **Critic/Red-Team** adversarial node, the **diagnose→solve→validate** worker roles (deer-flow only researches), the **loop-until-confident** controller, the **simulator-in-the-loop** gate, and the **Memory** learning node.

2.3 The LangGraph state-graph

```
g = StateGraph(SolveState)
for n,f in [("coordinator",coordinator),("planner",planner),
           ("graph_builder",graph_builder),("correlator",correlator),
           ("rca",root_cause),("solgen",solution_gen),
           ("validator",validator),("critic",critic),
           ("reporter",reporter),("human_gate",human_gate),
           ("memory",memory)]:
    g.add_node(n,f)

g.set_entry_point("coordinator")
g.add_edge("coordinator","planner")
g.add_edge("planner","graph_builder")    # planner emits solve-DAG;
                                         # default linear team
g.add_edge("graph_builder","correlator")
g.add_edge("correlator","rca")
g.add_edge("rca","solgen")
g.add_edge("solgen","validator")
g.add_edge("validator","critic")

def after_critic(s):                    # loop-until-confident (CORE
    controller)
    if s["loop_count"] >= MAX_LOOPS:    return "reporter"    # escalate w/
        low conf
    if any(c.severity=="refutes_cause" and c.open for c in s["critiques"]):
        return "rca"    # cause refuted → re-diagnose
    if any(c.severity=="refutes_solution" and c.open for c in
        s["critiques"]):
        return "solgen"    # fix refuted → re-solve
    if s["confidence"] < CONF_THRESHOLD: return "solgen"
    return "reporter"
g.add_conditional_edges("critic",after_critic,
    {"rca":"rca","solgen":"solgen","reporter":"reporter"})

g.add_edge("reporter","human_gate")
def after_gate(s):
    return {"approved":"memory","rejected":END,"revise":"planner"}
        [s["gate"]]
g.add_conditional_edges("human_gate",after_gate,
    {"memory":"memory","planner":"planner",END:END})
g.add_edge("memory",END)

app = g.compile(checkpointer=saver, interrupt_before=["human_gate"]) #
    deer-flow HITL
```

`interrupt_before=["human_gate"]` is the deer-flow human interrupt: the flow pauses, the operator inspects the decision card, and resumes by setting `state.gate`. The `critic→rca/solgen` back-edges are the loop; `loop_count` bounds it.

2.4 Swarm trace — Zargha water cascade

Pack = `water_pack` (mains topology ontology, hydraulic propagation rules, SCADA/911 connectors, valve/tanker interventions, EPANET simulator).

1. **Coordinator** ingests SCADA pressure drops, a hospital-load feed, traffic, and the 911 stream (+320%); selects `water_pack`; routes to Planner.

2. **Planner** emits solve-DAG: build→correlate→rca→solve→validate, stop when `confidence≥0.8` and zero open refutations.
3. **Graph-BUILDER** resolves entities (`PIPE-ZN-44` , `HOSP-3` , `PSTATION-7` , `CALLCTR`) and lays typed edges (`feeds` , `pressure_dep` , `serves`) from `pack.ontology` .
4. **Correlator** clusters the pressure drop, hospital strain, gridlock, and 911 spike into **one** incident via `pack.rules` co-occurrence + topology adjacency.
5. **Root-Cause Analyst** runs upstream graph traversal: the 911 surge and hospital strain are **leaf symptoms**; the only node upstream of all branches is `PIPE-ZN-44` . Sets `root_cause = rupture@PIPE-ZN-44 (0.86)` , evidence = the dependency paths.
6. **Solution-Generator** queries `pack.interventions[pipe_rupture]` → {isolate via valves `V-12/V-15` , deploy 4 tankers to `HOSP-3`, reroute traffic}. Emits `SolutionPlan` with resources/authority/time.
7. **Validator** runs EPANET via `pack.simulator` : isolation restores pressure to `HOSP-3` (Δ risk -0.62 vs no-action), 2nd-order harm bounded (downstream zone loses non-critical supply 3h). Passes.
8. **Critic / Red-Team round**: objects "could be a pump failure at `PSTATION-7` , not the pipe." Tagged `refutes_cause` → routes back to **RCA**. RCA checks SCADA flow at the station (nominal) and refutes the alternative; hypothesis survives, confidence $\uparrow$ $0.86 \rightarrow 0.91$. Critic re-runs, raises `refutes_solution` : "valve `V-15` isolation strands a dialysis clinic." Routes to **Solution-Generator**, which swaps to `V-12+V-19` . Validator re-confirms. Critic finds no open refutations → Reporter.
9. **Reporter** emits the **DecisionCard**: root cause, the validated fix, Δ risk, feasibility, bounded harm, confidence 0.91, full evidence lineage (§4 Validation).
10. **Human-Gate** (interrupt): duty officer approves → `gate="approved"` .
11. **Memory** records predicted vs realized Δ risk after execution, nudging `pack` priors for future pipe-rupture cases.

Swap `water_pack` for `outbreak_pack` or `grid_pack` and the **identical** node graph diagnoses a contamination source or a tripped substation — proving core/pack separation (§5 Multi-Domain Proof). Engine details in §1 Architecture; solution synthesis in §3 Solution Engine; tech stack in §6.

3. General Root-Cause → Solution Engine (pack-driven)

This engine consumes a **confirmed RootCause** (the node/edge the §2 swarm isolated as the cascade origin — e.g. `PIPE-ZN-44` , not the loud 911 surge) plus the live dependency graph, and emits **ranked, evidence-bearing candidate solutions**. The **algorithm is fixed and domain-agnostic**; all domain knowledge lives in the active **Domain Pack's Intervention Library** (a pack file). Swapping `water.pack` for `outbreak.pack` changes the templates, *not* this code.

3.1 The generic `InterventionTemplate` (pack-supplied)

A pack declares an array of templates. The core treats each as an opaque, typed record:

```

# entry in <pack>/interventions.yaml – DATA, not code
- id: isolate_trunk_rupture
  cause_types: [pipe_rupture, main_break]           # which RootCause kinds
  this answers
  target_selector:                                  # how to bind to graph
    nodes/edges
    kind: node
    match: { type: pipe, role: trunk }
    scope: root                                     # root | controllers |
    neighborhood
  preconditions:                                    # feasibility predicates
    over graph+pack
    - has_isolation_valve(target)
    - crew_available(authority="ZN_Water_Ops")
  cdg_effect:                                       # solution = a GRAPH
    MUTATION
    op: cut_propagation
    on: out_edges(target)
    risk_multiplier: 0.05                          # residual flow after
    isolation
  required_resources: { crews: 1, valves: 2 }
  responsible_authority: ZN_Water_Ops
  lag: 25m                                          # time-to-effect
  durability: 0.9                                 # how long effect holds
    (0-1)
  root_cause_acting: true                          # TRUE only if it acts ON
    the root cause

```

Every field is generic. `cdg_effect` is the crux: a **solution is expressed as a candidate mutation of the Causal Dependency Graph (CDG)** — cut an edge, damp a propagation weight, inject a supply node, raise a node's capacity. Validation (§4) re-simulates the graph under this mutation. `root_cause_acting` is the gate that lets us **reject symptom-targeting candidates**: a template that merely adds hospital beds or rerouting traffic has `root_cause_acting: false` and is filtered before scoring (kept only as a labeled palliative, never ranked as a solution).

3.2 Generic candidate generation

Three fixed strategies, all parameterized by the pack:

1. **Root-node templates** — every template whose `cause_types` contains the `RootCause.type` and whose `target_selector(scope=root)` binds to the root node. Direct countermeasures.
2. **Controller templates on the cascade** — compute **edge betweenness** on the active cascade subgraph; the highest-betweenness edges are the choke points propagating harm. Bind templates with `scope=controllers` to the nodes governing those edges (valves, isolation switches, checkpoints). These cut the cascade even when the root is slow to neutralize.
3. **Bundling** — greedily combine a root-acting template with controller/neighborhood palliatives into composite candidates, so long as resources/authorities don't conflict. A bundle's `cdg_effect` is the *composition* of its members' mutations. This is how the engine produces "isolate **and** bypass **and** tanker" rather than three rival singletons.

3.3 Generic scoring contract

Each candidate is scored by one fixed formula:

```
score = w_r·ΔRisk_attributed - w_c·cost - w_h·secondOrderHarm
score *= feasibility           # multiplicative gate ∈ [0,1]
```

- **ΔRisk_attributed** — drop in the incident's risk, *attributed to the root cause*, from re-simulating the graph under `cdg_effect` vs no-action (the §4 simulator returns it).
- **cost** — normalized `required_resources`.
- **secondOrderHarm** — new risk the mutation *creates* elsewhere (bounded; a VALID-SOLUTION criterion).
- **feasibility** — product of precondition checks × resource/authority/time availability.
Multiplicative, so any infeasible precondition zeroes the candidate — feasibility can't be "bought" by a large ΔRisk.

Weights `w_r`, `w_c`, `w_h` are pack-tunable; defaults favor risk reduction.

3.4 Pseudocode (fixed algorithm)

```
def generateSolutions(rootCause, graph, pack):
    casc = cascade_subgraph(graph, rootCause)
    bet = edge_betweenness(casc)
    choke_nodes = controllers_of(top_k_edges(bet))

    cands = []
    for t in pack.interventions:
        if rootCause.type not in t.cause_types:
            continue
        if t.target_selector.scope == "root":
            targets = bind(t.target_selector, [rootCause.node])
        elif t.target_selector.scope == "controllers":
            targets = bind(t.target_selector, choke_nodes)
        else:
            targets = bind(t.target_selector, neighborhood(graph,
                rootCause))
        for tgt in targets:
            if all(check(p, graph, tgt, pack) for p in t.preconditions):
                cands.append(Candidate(t, tgt,
                    mutation=t.cdg_effect.apply(tgt)))

    cands += bundle(cands, pack)          # composite, conflict-free

    # REJECT symptom-only candidates: keep palliatives labeled, rank only
    # root-acting
    ranked = []
    for c in cands:
        if not (c.template.root_cause_acting or
            c.in_bundle_with_root_actor):
            c.label = "palliative"; continue
        d = simulate_delta(graph, c.mutation, pack.simulator)    # §4
        feas = feasibility(c, graph, pack)
        c.score = (pack.w_r*d.dRisk - pack.w_c*c.cost
            - pack.w_h*d.secondOrder) * feas
        c.evidence = lineage(rootCause, c, d)                    #
        confidence + lineage
        ranked.append(c)
    return sorted(ranked, key=lambda c: c.score, reverse=True)
```

3.5 Worked

Zarqa water (water.pack). RootCause = PIPE-ZN-44 (pipe_rupture) . Root template `isolate_trunk_rupture` binds to the rupture; betweenness flags the trunk's downstream edges, pulling in `open_bypass_main` (controller) and `dispatch_tanker_supply` (neighborhood palliative, root-acting via the bundle). The bundle **isolate + bypass + tanker** wins: highest $\Delta\text{Risk}_{\text{attributed}}$ (cuts the rupture's propagation at risk_multiplier 0.05, restores pressure), feasible (valves + 1 crew + tankers under `ZN_Water_Ops`), bounded second-order harm. Candidates like "surge hospital capacity" or "reroute traffic" are `root_cause_acting:false` → labeled **palliative**, never out-ranked above the fix.

Outbreak (outbreak.pack), same code. RootCause = CLUSTER-7 (index_cluster) → winning bundle **isolate index cluster + surge supply** (quarantine the choke-betweenness transmission edge + dispatch supplies). Identical `generateSolutions` ; only

`interventions.yaml`, `target_selector` ontology, and the simulator differ. "Add ICU beds" is the symptom-targeting palliative the engine correctly refuses to rank as the solution.

See §4 for how each `cdg_effect` is simulated and gated into a VALID solution, and §5 for the full multi-domain proof.

4. General Validation & Safety (simulator-adapter)

Every candidate from §3 Solution Engine is a **hypothesis**, not a recommendation. This section is the domain-agnostic gate that turns hypotheses into validated solutions. **Core owns the gate; the pack owns the physics**. The core never knows what a pipe or a reproduction number is — it only calls a `SimulatorAdapter` the pack supplies, reads back a risk trajectory, and applies a fixed six-step rubric.

4.1 The `SimulatorAdapter` interface (pack-provided)

A pack provides one adapter implementing this contract. The core depends only on the contract.

```
class SimulatorAdapter(Protocol):
    def baseline(self, graph: CrisisGraph, horizon: Horizon) -> RiskTrajectory:
        """No-action counterfactual: risk(t) if we do nothing."""

    def apply(self, graph: CrisisGraph,
              intervention: Intervention, # {target_node, action, params,
              t_apply}
              horizon: Horizon) -> RiskTrajectory:
        """Counterfactual WITH the intervention applied at t_apply."""

    def fidelity(self) -> float # 0..1, declared by pack; scales
        confidence
    def supports(self, action_type: str) -> bool

@dataclass
class RiskTrajectory:
    t: list[float]
    risk_total: list[float] # incident-level risk over time
    per_node: dict[NodeId, list[float]] # needed for second-order harm
    feasible: bool # sim rejected as physically
                impossible?
```

Binding (core → pack): water → WNTR/EPANET (hydraulic solve, re-run with valve/pump op applied); system-dynamics → PySD/BPTK (stock-flow re-integration with a changed rate); agent-based → Mesa (re-run ensemble with changed agent rule). **Default fallback:** if a pack ships no high-fidelity simulator, the core uses the **graph cascade model** — the same propagation engine that built the dependency graph, run as a counterfactual by clamping the target node's state and re-propagating. Fallback declares `fidelity() ≈ 0.4` so confidence is discounted, never faked.

4.2 The fixed validation gate (domain-independent)

Each candidate runs all six steps; any hard failure → reject. The gate is pure core logic over `RiskTrajectory` objects.

```
GATE(candidate, graph, adapter):
    base = adapter.baseline(graph, H)
    cf    = adapter.apply(graph, candidate.intervention, H)

    # (1) counterfactual risk drop vs no-action
    drop = AUC(base.risk_total) - AUC(cf.risk_total)
    if drop <= MIN_DROP: reject("no risk reduction")

    # (2) root-cause-targeting check
    if candidate.target not in causal_controllers(graph, root_cause):
        reject("acts on symptom, not cause/controller")

    # (3) second-order-harm counterfactual (NET across ALL nodes)
    harm = sum(max(0, AUC(cf.per_node[n]) - AUC(base.per_node[n]))
               for n in graph.nodes)
    if (drop - harm) <= 0: reject("net harm: side effects exceed benefit")

    # (4) feasibility / authority / lag gate
    if not (resources_ok(candidate) and authority_ok(candidate)
           and candidate.lag <= H.window): reject("infeasible")
    if not cf.feasible: reject("simulator: physically impossible")

    # (5) confidence + lineage
    conf = w1*norm(drop) + w2*adapter.fidelity() + w3*evidence_strength
    candidate.lineage = {base, cf, root_cause_id, controller_path, sim_runs}

    return Pass(candidate, drop, net=drop-harm, conf)
```

- **(1)** uses area-under-curve so a fix that delays vs. truly suppresses is scored honestly.
- **(2)** is the symptom-firewall: `candidate.target` must lie on the controller set of the root cause (from §3). Treating a downstream symptom node fails here even if its local risk drops.
- **(3)** sums only *increases* per node, net against the benefit — bounds second-order harm globally, not just at the target.
- **(6) Abstain / escalate (via-negative)**: if **no** candidate passes, the core does **not** invent one. It emits `ABSTAIN` with the closest-miss diagnostics and escalates to human-in-the-loop. Doing nothing safely beats recommending an unvalidated act.

4.3 Acceptance tests for "valid solution" (pack-independent)

These assert on the rubric, not on any domain.

```

T1 risk-drop:      PASS  $\Rightarrow$   $AUC(cf) < AUC(base)$  by  $\geq MIN\_DROP$ 
T2 targets-cause: PASS  $\Rightarrow$   $target \in controllers(root\_cause)$ ; a symptom-
targeting
                    twin of the same candidate is REJECTED
T3 net-benefit:    inject a node whose harm > local benefit  $\Rightarrow$  candidate
REJECTED
T4 feasibility:    set authority=false OR lag>window  $\Rightarrow$  REJECTED
T5 lineage:        PASS  $\Rightarrow$  result carries baseline+cf runs, root_cause_id,
conf $\in[0,1]$ 
T6 via-negativa:   graph with no valid controller  $\Rightarrow$  ABSTAIN, never a guess
T7 fidelity-disc:  same candidate under fallback sim has conf < under hi-fi
sim

```

4.4 Worked Zarqa validation

Root cause = PIPE-ZN-44 rupture; controllers = upstream valves/pump (VALVE-ZN-7). Two candidates enter the gate against the WNTR adapter.

- **Candidate A — "treat the ER" (surge staff + divert 911):** target = HOSPITAL-Z1 , a *symptom* node. Fails **step (2)**: not in `controllers(PIPE-ZN-44)` . Even though local ER risk dips, the rupture keeps cascading — `base.risk_total` is essentially unchanged, so it would also fail **(1)**. **REJECTED** as symptomatic.
- **Candidate B — "isolate + reroute" (close VALVE-ZN-7 , repressurize from west main):** target $\in$ controllers. WNTR baseline AUC $\Rightarrow$ incident risk **54.3**; counterfactual with the valve op $\Rightarrow$ **~31.7** ($\Delta \approx 22.6 > MIN_DROP$) $\rightarrow$ **(1) PASS**. Step (3): west-main pressure rise nudges two nodes up by a small amount, net still strongly positive $\rightarrow$ **PASS**. Crews + valve authority available, lag < window $\rightarrow$ **(4) PASS**. Confidence high (hi-fi WNTR fidelity ≈ 0.9 + strong lineage). **VALIDATED**, with baseline/cf trajectories attached as evidence.

The same gate, unchanged, validates a PySD outbreak fix or a Mesa grid-cascade fix — only the adapter swaps (cross-ref §5 Multi-Domain Proof; engines/stack in §3, §6).

5. Multi-Domain Proof — the same engine, three packs

The claim of §1 Architecture and §2 Swarm is that the engine, the solver swarm, and the algorithms (entity-resolution, correlation, root-cause ranking, risk-cascade) are **domain-agnostic**. This section proves it operationally: we run the **identical** binary on three crises, changing **only** the loaded pack. Same `run()` call, same `RootCauseEngine` (§3 Solution Engine), same simulator-driven `Validator` (§4 Validation). The only delta is the `DomainPack` object.

```

# IDENTICAL across all three runs – nothing below is domain-specific.
brain = CrisisBrain(swarm=Swarm.default(), rce=RootCauseEngine(),
                    validator=Validator())

for pack in [load_pack("water"), load_pack("public_health"),
             load_pack("power")]:
    result = brain.run(case=ingest(pack.connectors), pack=pack)
    assert result.root_cause.is_cause_not_symptom    # §3 invariant
    assert result.solution.validated                # §4 invariant

# DomainPack – the ONLY thing that changes (see §1 for the interface).
DomainPack = {
    "ontology":      EntityEdgeSchema,    # node/edge types + resolution keys
    "rules":         PropagationRules,    # how risk flows along edges
    "connectors":    [Adapter, ...],      # raw signals -> typed nodes
    "intervention":  {cause_type: [Countermeasure, ...]},
    "simulator":     SimAdapter,          # validates candidate solutions
}

```

Pack A — Water (Zarqa pipe rupture)

Stage	Content
Signals / symptoms	911 calls +320% (LOUD), ER admissions +40% , traffic jams downtown, pressure drop on PIPE-ZN-44 (quiet)
Graph	PIPE-ZN-44 →supplies→ {ZONE-7, HOSPITAL-3}, ZONE-7 →feeds→ traffic/911 demand ; propagation rules flow pressure-loss risk downstream
Root cause	PIPE-ZN-44 trunk-main rupture — highest upstream causal score, earliest onset; 911/ER are downstream symptoms , not causes
Validated solution	isolate(valve V-44a/b) + bypass(ZN-43) + dispatch tankers→HOSPITAL-3 ; sim shows zone-risk ↓78% vs no-action, feasible in 90 min, bounded outage to ZN-7

Pack pieces used: water ontology (Pipe/Zone/Facility), hydraulic propagation rules, SCADA + CAD-911 connectors, pipe_rupture → {isolate, bypass, tanker} library, EPANET-style hydraulic simulator.

Pack B — Public Health (waterborne outbreak)

Stage	Content
Signals / symptoms	Clinic visits +260% (LOUD), school absenteeism +55% , ORS stockouts, faint chlorine-residual dip at WELL-12 (quiet)
Graph	WELL-12 →serves→ {DISTRICT-N} , DISTRICT-N →contains→ {CLINIC-A, SCHOOL-set} ; rules propagate exposure-risk along the water-source→population edges
Root cause	WELL-12 fecal contamination — case clusters geo-trace to its service area, onset precedes clinic surge; clinic/school load are symptoms
Validated solution	isolate(WELL-12) + chlorinate(supply) + surge(ORS→CLINIC-A) ; SEIR sim shows attack-rate ↓ vs no-action, feasible with district stock, second-order water-shortage bounded by tankering

Pack pieces used: epi ontology (WaterSource/District/Clinic/Cohort), exposure/transmission rules, HMIS + lab + WASH connectors, source_contamination → {isolate, chlorinate, ORS-surge} library, SEIR/compartmental simulator.

Pack C — Power Grid (feeder fault cascade)

Stage	Content
Signals / symptoms	Hospital backup-generator alarms (LOUD), water-pumping station offline, telecom tower on battery, feeder FDR-9 relay trip (quiet)
Graph	FDR-9 →energizes→ {SUBSTN-2} , SUBSTN-2 →powers→ {PUMP-STN, TELCO-T, HOSPITAL-3} ; rules propagate load-loss risk down the power-dependency edges
Root cause	FDR-9 feeder fault — single upstream node whose loss explains all downstream alarms; generator/pump/telecom alarms are symptoms
Validated solution	reroute(FDR-9→FDR-7) + shed(non-critical load) + prioritize(HOSPITAL-3) ; power-flow sim shows served-critical-load ↑, no thermal/voltage violations, feasible within switching limits

Pack pieces used: grid ontology (Feeder/Substation/Load), load-flow propagation rules, SCADA/AMI connectors, feeder_fault → {reroute, load-shed, prioritize} library, power-flow (DC/AC) simulator.

The decisive pattern

In all three, the **loudest** signal (911, clinics, generator alarms) is a downstream **symptom**, and the **quiet** upstream node (PIPE-ZN-44 , WELL-12 , FDR-9) is the true cause. The identical root-cause algorithm wins each time by ranking on **causal-upstream-ness + onset-precedence**, not on signal volume — see §3 for the ranking and §4 for the simulate-vs-no-action gate that certifies each fix.

Core vs Pack — the line that never moved

CORE (unchanged across A/B/C)	PACK (swapped per domain)
Graph substrate + entity-resolution engine (§1)	Entity/edge ontology + resolution keys
Correlation → incident clustering (§1)	Propagation/dependency rules
Root-cause ranking algorithm (§3)	Data connectors/adapters
Solver swarm: coordinator→planner→solver→validator (§2)	Intervention library (cause-type → countermeasures)
Validation harness + valid-solution criteria (§4)	Simulator adapter
Confidence scoring + evidence lineage	(domain thresholds as pack config)

The left column is one codebase compiled once; the right column is three declarative bundles. Adding a fourth domain (telecoms, finance-ops, supply chain) is a **pack-authoring** task, not an engine change — exactly the deer-flow-style reusable backbone targeted in §6 Tech Stack and scheduled in §7 Build Plan.

6. Tech Stack — Agent Frameworks + Engine Libraries

The crisis-solving brain is a general, deer-flow-style system built from two layers. The **agent-flow shell** is the domain-independent swarm backbone: a deer-flow-style LangGraph state graph (plan → execute → synthesize, with a human review gate and pluggable MCP tools) that we re-skin from research→reporting into diagnose→solve, plus the role-swarm, deep-research, low-code, and memory/runtime peers that inform its supervisor, validation, and durability behaviors. The **engine libraries** are domain-agnostic computational primitives — graph stores and traversal, entity resolution, stream correlation, anomaly detection, causal/root-cause analysis, graph ML, simulation, and optimization — that the shell's nodes call as tools. A new domain (water, grid, public health, etc.) adds only a **pack**: data connectors, an ontology, an intervention library, and a simulator adapter. The shell and the engine never change; the pack plugs in as the tools/ontology the nodes invoke.

Agent-Flow Shell (the swarm backbone)

Recommended backbone: deer-flow + LangGraph — the deer-flow-style state graph (plan → execute → synthesize, human review gate, MCP tools) running on LangGraph's durable, checkpointed, interrupt()-capable runtime. Everything else below is a reference, peer, or substrate that refines specific behaviors (supervisor fan-out, validate-and-revise, durability, authoring, memory).

Orchestration runtime (LangGraph family — the stateful flow engine)

Repo	~Stars	License	Role in the brain	Why
langchain-ai/langgraph	~33.4k	MIT	The runtime substrate of the flow engine: nodes = diagnostic/solver agents, edges = control.	Checkpointing gives durable case state; interrupt() gives the human-in-the-loop approval gate before committing an intervention; the State object holds the dependency graph / incidents / candidate solutions as they evolve. 540+ releases, very actively maintained.
DAGWorks-Inc/burr	~2.0k	Apache-2.0	Lower-abstraction alternative flow engine: actions = agent steps, state = the evolving case graph, transitions = control.	persist/rewind + telemetry UI map onto evidence lineage, replayable diagnosis, and operator inspection of why a root cause was chosen. Apache-incubating.
run-llama/workflows-py	~384	MIT	Event-driven flavor of the flow engine: each agent is a Step consuming a signal/incident Event and emitting the next.	The event bus naturally models the crisis as a stream of signals propagating through the dependency graph, with parallel fan-out across incidents.

Reference shell + deep-research peers (deer-flow deep-dive)

Repo	~Stars	License	Role in the brain	Why
bytedance/deer-flow	~70.1k	MIT	The general agent-flow BACKBONE we adapt: the coordinator/planner/research-team/reporter LangGraph graph with a human review gate and pluggable MCP tools, re-skinned research→reporting → diagnose→solve.	The domain pack the tools/ontology call; the graph itse changes. Note: th v1 plan-execute-s graph lives in tagc releases / git histc v2.0), since HEAD v2.0 SuperAgent t
langchain-ai/open_deep_research	~11.5k	MIT	Cleanest reference for the SOLVER SWARM's supervisor → parallel-subagent fan-out.	Root-cause invest decomposes a ca independent sub- investigations (per cause / per affecti subsystem) that r concurrently with context, then a sy ranks hypotheses parallel, context-is decomposition the domain proof nee Research Bench #
assafelovic/gpt-researcher	~27.4k	Apache-2.0	Reference for the VALIDATION / review loop in the valid-solution criteria.	Its Reviewer→Rev is a working critiqu revise loop mappi "validate the cand solution, then revi (root- cause/feasibility/s order-harm/sim-ri reduction) before : — the part deer-fl reporter lacks.
langchain-ai/deepagents	~23.6k	MIT	The reusable harness layer UNDER the swarm: planning + subagent spawning + working memory.	Build the brain's n deepagents' planning/filesyste middleware (the ": subagent" substr expose domain-pi connectors/simula tools its subagent on LangGraph.
stanford-oval/storm	~28.3k	MIT	Deep-research peer: multi-perspective evidence-gathering + grounded synthesis.	Discovering divers viewpoints and bu structured concep of a problem befo committing to cor

Repo	~Stars	License	Role in the brain	Why
				Co-STORM adds + human-in-the-loop. Mature/stable (last update 2025-09).

Role-based multi-agent (the solver swarm roles)

Repo	~Stars	License	Role in the brain	Why
crewAIInc/crewAI	~52.5k	MIT	Cleanest mapping onto solver-swarm roles: planner, analyst(s), solver, critic, reporter as declarative role/goal/backstory + tools.	Hierarchical process manager agent that delegates and validates. mirrors our coordinator/planner research-team support. Flows give the deterministic, gated path for the validation plan loop. No LangChain dependency.
FoundationAgents/MetaGPT	~68.4k	MIT	SOP-driven role assembly line: PM/Architect/Engineer/QA ≈ planner → analysts → solver → critic.	Its built-in QA Engine is a direct analog of validation/critic agent. "Code = SOP(Team structured-output maps to schema's evidence lineage + confidence scores. Moderate cadence.
ag2ai/ag2	~4.6k	Apache-2.0 (+ MIT for inherited AutoGen code)	Conversational role-orchestration layer: each role a ConversableAgent with a role-specific system message.	GroupManager/AutoGen handles supervisor selection coordinator); swarm HANDOFFS implement explicit role-to-role transitions (diagnose → solve → validate). formerly AutoGen;

Lightweight handoff/agent SDKs (minimal swarm runtime + tool/handoff primitives)

Repo	~Stars	License	Role in the brain	Why
openai/openai-agents-python	~26.8k	MIT	Minimal swarm runtime + tool/handoff primitives — the canonical, maintained implementation of the class.	Agents, Tools, Handoffs, Guardrails, Runner, Sessions, human-in-the-loop, sandbox agents, tracing, LiteLLM/100+ providers — the lean runtime when the full LangGraph backbone is more than a sub-swarm needs.
huggingface/smolagents	~27.6k	Apache-2.0	Lightweight code-acting agent + managed sub-agent variant.	CodeAgent runs in ~30% fewer steps. Note: LocalPythonExecutor is NOT a security sandbox — use E2B/Modal/Docker/Blaxel for untrusted code.
BrainBlend-AI/atomic-agents	~6.0k	MIT	Typed, composable end of the class — tool/handoff primitives via schema-aligned chaining.	Instructor + Pydantic foundation; schema-aligned input/output chaining and context providers fit the schema'd evidence-lineage requirement.
openai/swarm	~21.6k	MIT	The original reference definition of the class (handoff primitive).	DEPRECATED by README notice (replaced by the OpenAI Agents SDK) and effectively frozen — study it, don't build on it.

Autonomous general problem-solvers (open-ended planning+execution loop references)

Repo	~Stars	License	Role in the brain	Why
OpenHands/OpenHands	~75.5k	MIT	Reference for the robust execution + sandboxing substrate beneath the swarm.	Its event-stream/state model computer interface map onto connectors, graph computation simulator calls safely and ob: Software Agent SDK models brain as a composable library from one diagnosis to thousand. The sandboxed runtime is with pack's connectors and simulation execute in isolation.
FoundationAgents/OpenManus	~56.4k	MIT	Compact reference for the per-agent ReAct loop each node runs.	planning-agent + tool-call loop "think → select tool/connector observe → iterate"; run_flow. lighter-weight alternative to L DataAnalysis agent demonstrates Domain-Pack idea in miniature
kortix-ai/suna	~19.8k	Elastic License 2.0 (source-available, NOT OSI)	Reference for connector breadth + the three execution modes (on-demand, human-assisted with checkpoints, fully automated).	Broad connector layer (MCP/OpenAPI/GraphQL/HTTP) models how a pack's data context intervention/actuation adapts domain-independent core. C prohibits hosted/managed-simulation

Low-code agent-flow builders (visual authoring + ops surface)

Repo	~Stars	License	Role in the brain	Why
langflow-ai/langflow	~149k	MIT	Visual flow authoring + ops with native, bidirectional MCP as the integration spine.	An authored flow (or any pack tool) becomes an MCP endpoint our deer-flow/LangGraph swarm consumes directly — matching deer-flow's own MCP-tool design. Python components let pack authors drop in entity-resolution, correlation, root-cause, and risk-cascade engine calls, and the simulator adapter, while keeping the core untouched.
langgenius/dify	~143k	Modified Apache 2.0 (Dify OSS License)	Ops-grade authoring surface + LLMOps observability.	Branching/loops/aggregators map to root-cause → intervention → validate; code+HTTP nodes become pack connectors and simulator-adapter calls; observability/gateway gives the run-lineage + confidence-score "ops" half. Caution: no multi-tenant/SaaS use from source; logo/copyright must remain.
FlowiseAI/Flowise	~53.2k	Dual: Apache-2.0 core + Commercial (enterprise/)	LangChain-native authoring front-end for the swarm.	Tightest LangChain/LangGraph lineage of the builders — an authored agentflow mirrors to the same primitives our backbone uses; custom tool nodes are where pack connectors and the simulator adapter attach.
n8n-io/n8n	~190k	Sustainable Use License (fair-code, NOT OSI)	Ops + connector/data-plumbing layer more than cognitive core.	400+ integrations ARE the pack's data connectors/adapters — pulling sensor/SCADA/911/EHR/grid feeds, firing alerts, writing back interventions; deterministic nodes handle feasibility checks and human-in-the-loop gates.

Agent memory & durable runtime (cross-case learning)

Repo	~Stars	License	Role in the brain	Why
letta-ai/letta	~23.1k	Apache-2.0	Persistent state/learning across crisis cases.	Stateful agents with self-editing memory blocks (formerly MemGPT); SQL/Postgres-backed durability for case knowledge that accumulates across incidents.
mem0ai/mem0	~57.2k	Apache-2.0	Persistent cross-case knowledge retrieval + optional graph memory.	Multi-level user/session/agent memory; multi-signal retrieval (semantic + BM25 + entity boosting + temporal). Note: current default is single-pass ADD-only; graph memory is a configurable Neo4j-style option, not the headline path; no decay mechanism.

Engine Libraries (domain-agnostic)

These are the computational primitives the shell's nodes call. Bolded item per group is the recommended default pick. The water-distribution libraries (WNTR/EPANET/EPyT) at the end are an **EXAMPLE PACK** showing how a domain plugs in — they are NOT part of the core engine.

Graph databases & in-memory graph engines (the Crisis Dependency Graph store + traversal)

Repo	~Stars	License	Maps to	Why
networkx/networkx	~17k	BSD-3-Clause	Reference in-memory store + traversal substrate for the Crisis Dependency Graph.	Typed nodes + weighted directed edges (lag as an edge attribute); ancestors()/descendants() drive backward causal traversal for root-cause; weighted shortest-path finds the symptom→root causal path (911 spike back to PIPE-ZN-44).
Qiskit/rustworkx	~1.7k	Apache-2.0	Faster (Rust-core) traversal for the same graph model.	Same conceptual ancestors/descendants/shortest-causal-path ops, fast enough to run cascade propagation and repeated what-if traversals during simulation/validation without leaving Python. Not a literal NetworkX drop-in — distinct API.
memgraph/memgraph	~4.1k	BSL 1.1 (+ Memgraph Enterprise License)	Persistent, server-grade store when the graph outgrows one Python process.	Cypher variable-length + weightedShortestPath queries for ancestors/descendants; native vector indexes co-locate signal embeddings next to the graph. MAGE library (40+ algorithms).
cozodb/cozo	~4.0k	MPL-2.0	Single-file embedded store with durability + vectors + temporal queries.	Recursive Datalog expresses ancestors/descendants and shortest-causal-path directly; time-travel queries graph state at incident-onset vs now; HNSW vectors support entity resolution. Caution: ~1.5yr without a release, pre-1.0.

Entity resolution / record linkage (resolving heterogeneous agency references to one canonical node)

Repo	~Stars	License	Maps to	Why
moj-analytical-services/splink	~2.2k	MIT	Core engine for resolving agency references to one canonical asset/service/location.	Blocking keys cheaply cluster candidates (PIPE-ZN-44 vs "trunk main 44" vs "ZN-44 ductile"); the Fellegi-Sunter scorer outputs a probability each pair is the same real-world node. DuckDB (~1M records/min) + Spark/Athena backends.
dedupeio/dedupe	~4.5k	MIT	Active-learning fit when little labeled data exists.	Gazetteer/RecordLink modes match incoming signals against an existing canonical asset registry (the graph's node table) and return confident clusters; human labels only uncertain pairs.
J535D165/recordlinkage	~1.0k	BSD-3-Clause	Composable building block for fine pipeline control.	Custom geo/string/date comparators to match locations and assets across agency schemas; pluggable classifiers and built-in precision/recall metrics to validate the resolver.

Stream processing / complex-event correlation (ingest signals + stitch correlated events in time windows)

Repo	~Stars	License	Maps to	Why
pathwaycom/pathway	~63.2k	BSL 1.1 → Apache-2.0	Stitching the lagged cascade with incremental, consistent temporal joins.	Lagged dependency edges (pipe → PS-12 → R-3 → service) make signals inherently out-of-order; as-of/interval joins correlate a late hospital-strain signal back to the earlier pressure-drop window without full reprocessing, producing a consistently-updated incident view.
quixio/quix-streams	~1.6k	Apache-2.0	Kafka-native windowed correlation.	Windowed group-bys keyed by asset/zone correlate S1..S5 into one incident; RocksDB-backed stateful aggregations track per-window features (pressure delta, call-rate spike) feeding the root-cause engine. Actively maintained.
faust-streaming/faust	~1.9k	BSD-3-Clause	Lightweight Kafka agents → windowed Tables.	"Agents" subscribe to per-agency topics, accumulating each signal's contribution within a window; async model fits bursty inputs like the +320% 911 spike. (Maintained fork of the abandoned robinhood/faust.)
bytewax/bytewax	~2.0k	Apache-2.0	Event-time windowing + watermarks to correlate despite lag/late arrival.	Stateful joins keyed by asset/location bind pressure-drop, reservoir-depletion, 911-surge, and hospital-strain together; noise filterable in the same dataflow. Caution: stale/at-risk — core team stepped back ~May 2025.

Anomaly & spike detection (fire incident-spike candidates into the correlation loop)

Repo	~Stars	License	Maps to	Why
yzhao062/pyod	~9.9k	BSD-2-Clause	Core spike/anomaly detector for batch multivariate telemetry.	Each Signal node's recent feature window is scored; a high anomaly score fires an incident-spike candidate. 60+ detectors (classical → deep) with a uniform sklearn-style fit/predict; decision_function gives a continuous per-signal severity.
online-ml/river	~5.8k	BSD-3-Clause	The real-time/streaming detection path.	Scores each new telemetry point in O(1) with constant memory and updates online, so the brain flags a spike the instant PS-12 pressure collapses; drift detectors catch regime changes (sustained reservoir depletion).
arundo/adtk	~1.2k	MPL-2.0	Interpretable, per-asset rule-based spike rules.	Level-shift catches the 6.2→1.1 bar drop; persist/threshold catches reservoir depletion; seasonal flags the 911 surge against its daily profile — clean evidence/lineage for root-cause and validation. Caution: dormant since 2020.
linkedin/luminol	~1.2k	Apache-2.0	Bridges spike detection into incident correlation.	After detecting the pressure-drop anomaly, correlate() ranks which other signals move together and with what lag — a cheap first pass at stitching S1..S5 while rejecting uncorrelated noise. Caution: effectively dormant since 2023.

Causal inference & root-cause analysis (the root-cause engine)

Repo	~Stars	License	Maps to	Why
py-why/dowhy	~8.1k	MIT	The core of the root-cause engine.	Feed the typed Crisis Dependency Graph to DoWhy-GCM as the causal graph; attribute_anomalies scores each node's contribution so PIPE-ZN-44 outranks the loud ER/911 symptoms; distribution_change handles "what changed"; the refutation/falsification API gives the validity/confidence checks the valid-solution criteria demand.
py-why/causal-learn	~1.6k	MIT	Backstops graph construction when edges/lags are uncertain.	Run PC/FCI/Granger over agency time-series to learn or validate causal edges, then hand the DAG to DoWhy-GCM; FCI's latent-confounder handling flags that fuel-price noise is not causally linked to the cascade.
phamquillan/RCAEval	~149	MIT	Validation/benchmarking harness + swappable RCA baselines.	Score our cause-ranking (does the true root cause land at top-1?) against 15 published methods on labelled cascades; lift specific causal-inference RCA implementations into the engine.
salesforce/PyRCA	~555	BSD-3-Clause	Turnkey blueprint + ready localizers for the localization step.	Bayesian-inference / random-walk over a causal graph map almost 1:1 onto walking the dependency graph backward from symptom nodes to the originating asset; dashboard prototypes the operator view.

Repo	~Stars	License	Maps to	Why
				Caution: ~2.5yr dormant.

Graph ML / GNN / network analysis (learned propagation weights + embeddings)

Repo	~Stars	License	Maps to
pyg-team/pytorch_geometric	~23.8k	MIT	Core engine for learned propagation weights and embeddings.
benedekrozemberczki/pytorch_geometric_temporal	~3.0k	MIT	Learns time-lagged propagation dynamics + forecasts cascades.
dmlc/dgl	~14.3k	Apache-2.0	Alternative/complement to PyG with stronger heterogeneous-graph ergonomics + scaling.

Repo	~Stars	License	Maps to

Simulation: system-dynamics / agent-based / discrete-event (prove an intervention halts the cascade)

Repo	~Stars	License	Maps to	Why
projectmesa/mesa	~3.7k	Apache-2.0	Run the cascade as autonomous, interacting agents.	PS-12, R-3, WATER-ZN, HOSP-ZN-1, JUNC-7, PSAP-911 and the population each become agents whose state evolves over ticks; simulate WITH vs WITHOUT a candidate intervention and measure whether the cascade halts and the risk index drops. NetworkGrid maps onto the dependency graph; mesa-geo adds GIS agents.
SDXorg/pysd	~451	MIT	Continuous physics as stocks and flows.	Reservoir R-3 volume (stock) driven by PS-12 throughput (flow); inject an intervention as a parameter change (restore pressure 1.1→6.2 bar) and integrate forward to show trajectories with vs without the fix. Imports Vensim/XMILE models.
salabim/salabim	~393	MIT	Discrete, resource-constrained parts of the crisis.	Tanker trucks competing for the JUNC-7 point (limited-capacity resource), 911 calls (queue + servers), repair-crew dispatch/lag; compare KPIs (wait times, time-to-restore) capturing feasibility/lag and second-order congestion. Pure-Python, zero deps.
transentis/bptk_py	~32	MIT	The intervention engine's scenario harness.	Define a baseline (no action) and multiple intervention scenarios (repair, isolate-and-reroute, tanker surge), run them through one framework, and diff outputs to rank candidates by risk-index reduction; hybrid SD+ABM keeps continuous hydraulics and discrete responders in one model.

Optimization / resource allocation / network flow (intervention selection)

Repo	~Stars	License	Maps to	Why
google/or-tools	~13.5k	Apache-2.0	The primary intervention-selection optimizer.	Encode candidate interventions (open/close valves, dispatch N tankers, reroute supply, schedule repair crews) as decision variables and resource/authority/budget/time limits as constraints; objective = maximize cascade/risk reduction at minimum cost. CP-SAT handles discrete bundle-selection; built-in min-cost-flow / VRP handle tanker routing and re-supply allocation.

EXAMPLE PACK — Water-distribution & infrastructure modeling (DOMAIN-SPECIFIC, not core)

These illustrate one domain pack's simulator adapter. A different domain (grid, epidemiology, logistics) swaps in its own equivalents; the shell and the engine above are untouched.

Repo	~Stars	License	Maps to	Why
USEPA/WNTR	~433	Revised BSD (3-clause)	The physics core of the water pack + ground-truth simulator that VALIDATES candidate fixes.	Model PIPE-ZN-44 as a pipe, PS-12 as a pump, R-3 as a tank, WATER-ZN as junctions; a rupture is a WNTR leak/break event and the pressure drop falls out of the pressure-dependent-demand solve. Re-run with a proposed intervention to measure restored service, recovered resilience index, and second-order effects. <code>wn.to_graph()</code> plugs into the Crisis Dependency Graph.
OpenWaterAnalytics/EPANET	~388	MIT	Low-level reference hydraulic solver under the water pack.	Canonical .inp network format and the toolkit calls that compute node pressures / pipe flows; the authoritative engine WNTR/EPyT call into, for raw/fast scriptable evaluations inside the intervention-ranking loop.
OpenWaterAnalytics/EPyT	~74	EUPL-1.2	Programmable bridge between EPANET and the agent swarm.	Fine-grained scriptable read/write control (close a valve, change a pipe's status/diameter, adjust demands for tanker dispatch) and immediate re-simulation — ideal for solver agents enumerating and testing candidate interventions in a tight loop against the authoritative solver.

Recommended minimal stack

The smallest buildable crisis brain is: a **deer-flow-style shell on LangGraph** (plan → execute → synthesize, human review gate, MCP tools — adding gpt-researcher's Reviewer→Revisor loop as the validation gate and open_deep_research's supervisor fan-out for parallel root-cause sub-investigations) calling, as tools, six engine primitives — **NetworkX** as the Crisis Dependency Graph store and traversal substrate, **Splink** for resolving heterogeneous agency references to canonical nodes, **DoWhy** (DoWhy-GCM) as the causal root-cause engine that ranks PIPE-ZN-44 above the loud symptoms, **PyOD** (or River for

streaming) for the spike detection that fires incident candidates, **Mesa** (or WNTR when the domain is water) as the simulator that proves an intervention halts the cascade, and **OR-Tools** for intervention selection — plus exactly one domain pack (the WNTR/EPANET/EPyT water pack is the worked example) supplying the connectors, ontology, intervention library, and simulator adapter. Everything heavier (Memgraph/rustworkx for scale, Pathway/Quix for live streaming correlation, PyG for learned propagation weights, Letta/mem0 for cross-case memory, Langflow/Dify for visual authoring) is an upgrade swapped in without touching the shell or the pack contract.

7. Build Plan, Adopting deer-flow, and Adding a Domain in N Steps

This section is the concrete construction sequence. It maps each build step to a **component category** (the §6 Tech Stack picks the actual repo), names the **acceptance gate** on the Zarqa case, and closes with the "add a domain in N steps" checklist. Invariant: the **core** (engine + swarm) is domain-blind; everything domain-specific lives in a **Domain Pack** loaded at runtime.

(a) Adopt/adapt deer-flow as the swarm shell

Layer on `deer-flow` /LangGraph rather than fork — keep it as a pinned dependency, vendor only the node prompts we override.

deer-flow node	Action	Becomes
Coordinator	Reuse	Intake: classify case, load matching Domain Pack
Planner	Reuse (reprompt)	Diagnose/solve plan over graph state
Human-in-the-loop gate	Reuse	Plan + intervention approval
MCP tool layer	Reuse	Connectors, simulator, graph queries as tools
Researcher + Coder (research team)	Replace	Diagnose team (correlator, root-cause) + Solve team (intervention)
Reporter	Replace	Cockpit/insight emitter
—	Add	Critic (challenges root-cause + plan) and Simulator-Validator (runs §4)

State is one shared `CaseGraph` object on the LangGraph blackboard; nodes read/write it. See §2 for the swarm topology and §3/§4 for the new nodes' contracts.

(b) Core build order (each step: category → gate)

1 Graph store + canonical model	[graph-db + schema]	gate: load
PIPE-ZN-44 + neighbors, traverse deps		
2 Ingestion / Entity Resolution	[ER / dedup engine]	gate: 911-surge
feed + SCADA + GIS → one entity set, no dup pipe		
3 Correlation	[event-correlation]	gate: cluster
rupture+hospital+traffic+911 into ONE incident		
4 Root-cause	[causal/RCA engine]	gate: rank
PIPE-ZN-44 as cause; 911/hospital scored as symptoms		
5 Risk cascade	[propagation engine]	gate: forward-
sim no-action risk over dep graph		
6 Intervention engine	[planner/rule + retrieval]	gate: cause-
type "trunk-rupture" → ranked countermeasures		
7 Simulator-adapter validation	[sim harness, \$4]	gate: chosen
fix reduces incident risk vs no-action		
8 Swarm wrapper	[deer-flow/LangGraph]	gate: nodes 1-
7 run as one graph, human-gated		
9 Cockpit / insight	[UI + lineage view]	gate: render
root cause, fix, confidence, evidence chain		

Steps 1–7 are libraries with hard interfaces; step 8 only orchestrates them; step 9 only reads outputs. Nothing above names "water" — Zarqa enters solely through a loaded pack. Each gate is an automated test in CI against the Zarqa pack fixture.

(c) MVP cut-line

MVP = **one end-to-end diagnose→validated-solve loop on one pack** (water/Zarqa). In scope: steps 1–9, single correlation strategy, rule-based root-cause + retrieval intervention, one simulator adapter, one human gate, Critic as a single review pass. Out of scope: multi-pack hot-swap UX, learned ranking, parallel incident handling, multi-simulator ensembles. Done = Zarqa case yields {root_cause: PIPE-ZN-44, intervention, sim_delta_risk<0, confidence, lineage} passing all nine gates with a human approval in the loop.

(d) Add a new domain in N steps (zero core changes)

A Domain Pack is a directory the core discovers and loads. Authoring checklist:

```
pack/  
  manifest.yaml      # 1. id, version, case-classifier hints  
  ontology.yaml     # 2. entity types + edge types (maps to canonical  
model)  
  rules.yaml        # 3. dependency/propagation + causal priors  
  connectors/       # 4. adapters → canonical events (implement Connector  
iface)  
  interventions.yaml # 5. cause-type → countermeasures (+ feasibility,  
cost)  
  simulator.py      # 6. Simulator adapter (implement run(scenario)→risk)  
  fixtures/         # 7. golden case + expected root_cause for CI gate
```

N = 7 steps: (1) declare manifest; (2) map ontology onto the canonical entity/edge model; (3) write dependency + causal-prior rules; (4) implement connectors to the canonical event

schema; (5) populate the intervention library keyed by cause-type; (6) wrap the domain simulator behind the `Simulator` interface; (7) ship a golden fixture. Drop the dir in `packs/`, register, run — the same engine and swarm now diagnose and solve that domain. §5 demonstrates this with a public-health-outbreak and a power-grid pack reusing the identical core, proving the boundary holds.

Each interface (Connector, Rule, Intervention, Simulator) is a thin contract; a pack that satisfies the contracts cannot require touching steps 1–9. If a new domain *needs* a core change, that is a core bug — the missing generality goes into the engine, never the pack.

Milestone table

M	Milestone	Steps	Gate	Pack
M0	Substrate	1–2	Entities resolved on graph	water
M1	Diagnose	3–4	Root cause = PIPE-ZN-44	water
M2	Risk + Solve	5–6	Ranked interventions emitted	water
M3	Validate	7	Sim shows risk reduction	water
M4	MVP loop	8–9	Full Zarqa loop, human-gated	water
M5	Critic + Validator	swarm	Bad root-cause rejected	water
M6	Multi-domain proof	pack-only	Same core solves 2nd + 3rd pack	health, power

M4 is the cut-line; M6 is the thesis (core unchanged across packs). Repo choices per category come from §6.
